

Sequence to driver

A sequence describes what to send, it builds sequence items and hands them to a sequencer

Items, start and finish, get and done

- The seq item is the transaction object, the currency between sequence and driver
- In the sequence body you create an item
- The driver does not push
- Sequences never wiggle pins directly, that is the driver's job
- Multiple sequences can share one sequencer; arbitration decides who goes next

Browser lab

Digital Design and Verification Platform

HomeTools

Home / Tools / Sequence → driver flow

INTERACTIVE TOOL

Sequence → driver flow

Follow one **seq_item** through sequence → sequencer → driver → vif → DUT. Starter:
UART byte item already at the sequencer, ready to step.

Starter example: UART byte 0xA5 sits on the sequencer — Step to hand it to the driver, then vif, then DUT.

Load starter example

Challenges 0 / 22 cleared

Quiz: path: The stimulus handoff path is...

☐ sequence → sequencer → driver → vif → DUT

☐ DUT → driver → sequence only

☐ scoreboard → factory → DUT

☐ report → build → connect

Show hint

Check

Next

Idle

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

Core ideas

seq_item

Transaction object — the currency between sequence and driver.

sequencer

Hands items to the driver; sequences do not drive pins.

driver

get_next_item → pin protocol on the vif.

DUT

End of the stimulus path — sees the driven signals.

Lab

SCENARIO

ITEM DATA

Sequence flow lab starter

learn_uvm2017 — uvm seq flow

Real UVM literacy

```
wsl - module09-uvms-seq-flow/examples/seq-flow-sketch

# Track A - real Unix
# real shell session (Track A - sequence → driver flow)

$ pwd
/mnt/d/proj/designs/learning/courses/learn_uvm2017/module09-uvms-seq-flow

$ ls examples/seq-flow-sketch/
flow.txt

$ cat examples/seq-flow-sketch/flow.txt
UVM sequence → driver flow sketch (literacy)

Path: sequence → sequencer → driver → vif → DUT

seq_item - transaction object (extends uvm_sequence_item)
fields: address, data, length, _ per protocol

Sequence body (describes WHAT to send):
req = MyItem::type_id::create("req");
start_item(req);
// randomize or assign fields
req.data = 8'hA5;
finish_item(req);           // hand off → sequencer → driver

Sequencer - arbitration + TLM to driver
multiple sequences can share one sequencer
finish_item pushes item toward driver side

Driver run task (describes HOW on pins):
forever begin
    seq_item_port.get_next_item(req); // pull from sequencer
    // translate req fields → vif signals (protocol)
    vif.tx ≤ req.data;
    @(posedge vif.clk);
    seq_item_port.item_done();        // release sequencer
end

Rules:
sequences do NOT wiggle pins - driver owns the vif
always item_done after get_next_item
connect: driver.seq_item_port * sequencer.seq_item_export

Common mistakes:
driving vif from sequence body
forget item_done → sequencer stalls
assuming finish_item means DUT already saw the transaction

$ grep -n "start_item\|finish_item\|get_next_item\|item_done"
.../..../learn_uvm2017_sv_verilator/module08/tests/uvms_tests/test_utilities_uvm.sv | head -10
80:         seq_item_port.get_next_item(txn);
83:         seq_item_port.item_done();
```

Real shell — sequence flow sketch

Pitfalls to watch

- Do not drive pins from the sequence, that bypasses the driver and breaks reuse
- Do not forget item done after get next item or the sequencer stalls waiting for you
- Do not assume finish item means the DUT saw it yet, the driver still has to run
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, step from sequencer to DUT and name each stage
- On real UVM, sketch the five-stage path with one example transaction
- When you are ready, take the short quiz, then continue to objections in the next module